

# Backend Reference — Resolvers, Consumers, and Shared Helpers

Function-by-function walkthrough of everything under `src/resolvers/`. Each entry says **what it does, who calls it, which auth it uses, and why it looks the way it does**, with short code snippets where the code itself is the best explanation.

Conventions used below:

- **Auth** — `asUser()` = acts with the caller's own Jira permissions; `asApp()` = acts as the app's service account. `useAppAuth` in code usually means "caller is unlicensed (portal customer), so their own session has no Assets access — act as the app instead."
- **Sync** = normal `invoke()` -backed resolver (25 s hard limit). **Consumer** = queue-triggered background function (own `timeoutSeconds`).

## 1. `shared.js` — the core library

Everything else imports from here. Read this file first when debugging.

### Constants & keys

```
export const CONFIG_KEY = 'assets-schema-config'; // the one admin config object
export const jobKey      = (id) => `asset-load-job:${id}`;
export const csvImportJobKey = (id) => `csv-import-job:${id}`;
export const exportJobKey   = (id) => `export-job:${id}`;
export const exportJobChunkKey = (id, i) => `export-job:${id}:chunk:${i}`;
```

| Constant                           | Value       | Meaning                                                                        |
|------------------------------------|-------------|--------------------------------------------------------------------------------|
| ASSET_LIST_CACHE_TTL_MS            | 60 000      | Per-user cached asset list is trusted for 1 minute                             |
| DEFAULT_PAGE_SIZE / MAX_PAGE_SIZE  | 10 / 10     | Rows hydrated per "Load more" click                                            |
| DEFAULT_USER_ASSET_LIMIT / ceiling | 500 / 5 000 | Admin's per-user cap and its sanity bound ( <code>clampUserAssetLimit</code> ) |
| ASSETS_API_PAGE_SIZE               | 100         | Page size against the Assets AQL endpoint                                      |
| ASSETS_API_TOTAL_CAP               | 1 000       | <b>Platform</b> ceiling per AQL query — one rule can never return more         |
| CACHE_MAX_ASSETS                   | 1 000       | Above this, skip the KVS cache write entirely (128 KB safety)                  |

## Small helpers

```
export const escapeAqlValue = (v = '') =>
  String(v).replace(/\\/g, '\\\\').replace(/"/g, '\\"');
```

**Every user-controlled value that ends up inside an AQL string goes through this.** It escapes backslashes and double quotes so a display name like Jane "JD" Doe can't break out of its quoted position. If you build AQL anywhere new, use it.

```
export const getAttrValue = (attribute, raw = false) =>
  attribute.objectAttributeValues?.map((v) =>
    raw ? v.status?.id || v.referencedObject?.id || v.value || ''
      : v.displayValue || v.value || v.referencedObject?.label || v.user?.displayName || ''
  ).filter(Boolean).join(', ') || '';
```

Assets attribute values are polymorphic (status objects, referenced objects, users, plain values). `raw=false` gives the human-readable string for display; `raw=true` gives the underlying id/value — the shape write APIs expect, which is what makes the CSV importer's "did anything actually change?" comparison possible.

Other one-liners: `normaliseAssets` (the Assets API returns results under three different key names depending on endpoint version — this normalizes), `getAttributeId` (same story for attribute ids), `deriveAttributeType` (maps Assets' numeric type codes to `'object' | 'status' | 'date' | 'select' | 'text'`), `parseOptions` (comma-string → array), `isUnlicensedCaller` (`context.accountType ∈ customer/anonymous/unlicensed`), `resolveUser` (profile fetch with app/user auth, returns an empty-fields fallback rather than throwing).

## AQL construction

```
export const buildAqlFromRow = (row, schemaId, user) => {
  // one admin rule → one complete AQL query
  // e.g. { attribute: "Owner", operator: "=", userField: "displayName" }
  // → objectSchemaId = 3 AND Owner = "Jane Doe"
  // with viaReference:
  // → objectSchemaId = 3 AND object HAVING outboundReferences(Owner = "Jane Doe")
}
```

`userField` selects which of the *verified* caller identity fields gets substituted (`accountId / displayName / email / literal currentUser()`). If the chosen field is empty on the user's profile, the rule yields `null` (rule skipped).

```
export const buildAqlCandidates = (aqlRows, schemaId, user, includeCurrentUser, filterAql) => {
```

Maps every rule through `buildAqlFromRow`, drops nulls, and — when a `filterAql` string is supplied — appends `AND <filterAql>` to every candidate. Two edge behaviors to know: rules configured but none resolvable (empty profile fields) returns a deliberate never-matching query (`Name = "__no_match__"`) rather than falling open; no rules configured returns `[]` (nothing to search).

```
export const buildFilterAql = (filters, schemaId) => {
```

Converts the frontend filter state into one AQL condition string:

- `nameQuery` → `(Name like "%q%" OR Key like "%q%")`
- `date attrs` → `"Attr" >= "from" / <= "to"` (ISO strings compare correctly)
- **`status attrs` → `exact =`**, not `like` — status attributes are reference fields under the hood and `LIKE` silently returns zero rows (this caused a "filter applies then clears itself" bug; the comment in code tells the story)
- `select/text` → `like "%v%"` (with multi-token OR for comma-separated selects)
- **numeric-only attribute names are skipped with a warning** — a bare number means the frontend failed to resolve an `attributeld` to a name, and letting it through would 400 *every* ownership candidate, not just this filter.

## Workspace & search

```
export const getWorkspaceId = async (useAppAuth = false) => {
```

Assets REST paths all need a workspace id. Tries `/rest/servicedeskapi/assets/workspace` then `/jsm/assets/workspace`; throws if neither answers. Callers cache the result per request — call it once, pass it around.

```
export const searchAssets = async (workspaceId, aql, useAppAuth = false) => {
```

Fetches **all** pages of one AQL query up to the 1,000-object platform cap. Page 1 is fetched alone (it reveals `total`); every remaining page is then fired **in parallel**. This replaced a sequential loop that, under multiple candidates × 10 pages, blew the 25-second `invoke()` ceiling (surfacing in `forge tunnel` as `ERR_IPC_CHANNEL_CLOSED`). A failed page logs a warning and is simply missing from the merge — tolerant by design.

`searchAssetsSinglePage` is the one-page variant used where 100 results is plenty (ownership verification probes, CSV key lookups, user pickers).

```
export const searchAllCandidates = async (candidates, workspaceId, useAppAuth) => {
```

Runs every ownership candidate concurrently and merges, de-duplicating by asset id. Returns `{ values, matchedAqls, hadAnySuccess, lastError }` — callers treat "some candidates failed but others succeeded" as success-with-warning, and only surface `lastError` when *nothing* succeeded.

```
export const verifyAssetOwnership = async (objectId, user, config, workspaceId) => {
  const candidates = buildAqlCandidates(config.aqlRows, config.schemaId, user, false);
  const { values } = await searchAllCandidates(candidates, workspaceId, true);
  return values.some((a) => String(a.id) === String(objectId));
};
```

The write-path gatekeeper: "would this object appear in this user's own asset list?" Always `asApp()` so it works for unlicensed customers too. Called by `updateAssetAttribute` before any PUT.

## Cache + list resolution

```
export const resolveAssetList = async ({ accountId, config, user, useAppAuth,
                                       forceFresh = false, filters = null }) => {
```

The single entry point for "get this user's assets." Orchestrates: cache check (skipped when `forceFresh` or when `filters` are present — the cache key is per-account only, so filtered results must neither be served from nor written to it) → `buildFilterAql` → `buildAqlCandidates` → `searchAllCandidates` → clamp to `maxUserAssetLimit` → cache write (unfiltered successes only, slim stubs, skipped above 1,000 items). Returns the list plus telemetry (`wasLimited`, `limitApplied`, `preLimitCount`) that drives the UI's "showing your first N assets" banner.

`assetListCacheKey(accountId)` is the cache key; **if you add any code path that changes a user's assets, delete this key** like `updateAssetAttribute` does.

## CSV helpers

```
export const parseCsvRows = (csvText) => {           // CSV text → { headers, rows }
export const matchCsvHeadersToAttributes = (headers, attributeDefs) => // → { matched, unmatched }
export const normalizeDateValue = (raw) => {         // '21/10/2025' | '45417.0001...' | ISO →
'YYYY-MM-DD'
export const fetchObjectTypeAttributeDefs = async (caller, workspaceId, objectId) => {
```

- `parseCsvRows` reads headers explicitly via SheetJS's raw header-row mode so a header-only CSV still reports its columns (rather than relying on `Object.keys(rows[0])`, which breaks with zero data rows).
- Header matching is **case-insensitive exact** — no fuzzy matching; a typo'd header lands in `unmatched` and is shown to the agent, never guessed at.
- `normalizeDateValue` handles the three date shapes seen in real files: ISO passthrough, `DD/MM/YYYY`, and bare Excel date serials (decoded via `XLSX.SSF.parse_date_code` — sidesteps the 1900 leap-year quirk).
- `fetchObjectTypeAttributeDefs` **keeps Name** (unlike the table's built-in filter) because Name is a real settable attribute and often the natural CSV unique key; `Key` is platform-generated and excluded.

## `buildAssetPayload` — hydration pipeline

```
export const buildAssetPayload = async ({ values, workspaceId, config, useAppAuth,
                                       ignoreHidden = false, typeIdsForColumns = null }) => {
```

Three stages, one per network round-trip class:

1. **Attribute definitions per object type** — for every distinct `objectId` in `values` (plus `typeIdsForColumns`, see below), fetch `/objecttype/{id}/attributes` and index by `typeId:attrId`.

Built-ins (Name/Key/Created/Updated) and admin-hidden attributes are excluded here, which is what makes hiding an attribute actually remove it from the payload.

2. **Status option labels** — status-type attributes reference status-type ids; this stage resolves them to human-readable labels once per distinct status type.
3. **Row mapping** — each raw asset becomes `{ id, label, objectKey, objectTypeId, objectTypeName, visibleValues, rawValues }` ( `visibleValues` = display strings keyed by `attributeld`; `rawValues` = ids/values for editing), and the union of stage-1 definitions becomes `columnsToShow`.

`typeIdsForColumns` exists for a subtle reason: when hydrating only page 1, types that appear later in the list wouldn't get column definitions — so tab headers and filters for those types would show raw ids. Callers pass the type ids of the *full* result so columns are complete from the start. `ignoreHidden: true` is used by the admin preview ( `getAssetsForUser` ) so admins see everything.

## 2. `resolvers/index.js` — portal-footer resolvers

| Resolver                           | Sync/Job          | Auth                                 | Purpose                                                                 |
|------------------------------------|-------------------|--------------------------------------|-------------------------------------------------------------------------|
| <code>getUser</code>               | sync              | asUser (agents) / asApp (unlicensed) | Display name for the greeting                                           |
| <code>getConfig</code>             | sync              | KVS only                             | Full config object (both portal + admin read it)                        |
| <code>getConfigVersion</code>      | sync              | KVS only                             | <code>{version, updatedAt}</code> — cheap poll to detect config changes |
| <code>startAssetLoadJob</code>     | sync<br>(enqueue) | context only                         | Start the heavy asset load; returns <code>{jobId}</code>                |
| <code>getAssetLoadJobResult</code> | sync              | KVS only                             | Poll a job's status/result                                              |
| <code>getUserAssetsPage</code>     | sync              | per-caller                           | Paginate/hydrate 10 more rows ("Load more")                             |
| <code>updateAssetAttribute</code>  | sync              | mixed (see below)                    | Inline edit with ownership verification                                 |
| <code>diagnoseCaller</code>        | sync              | per-caller                           | The "why do I see zero assets?" self-check                              |

## startAssetLoadJob / getAssetLoadJobResult

```
resolver.define('startAssetLoadJob', async ({ payload, context }) => {
  // ...identity checks...
  const unlicensed = isUnlicensedCaller(context); // captured HERE – context is
  const jobId = randomUUID(); // meaningless inside a consumer
  await kvs.set(jobKey(jobId), { status: 'pending', createdAt: Date.now() });
  //      ^ BEFORE the push – otherwise a fast consumer's 'done' could be
  //      overwritten by a late-arriving 'pending'
  await new Queue({ key: 'asset-load-queue' }).push({
    body: { jobId, accountId, limit, filters, unlicensed },
  });
  return { jobId };
});
```

This is the template for all three async jobs — the ordering comment and the identity capture are the two things not to break when copying it.

## getUserAssetsPage

Pagination within the (possibly cached) full list. The interesting optimization:

```
// Filtered (forceFresh) results come back from searchAssets with
// includeAttributes=true already – a stub carrying `.attributes` is
// already a full asset, so skip the extra per-object round trip
if (Array.isArray(stub.attributes)) return stub;
```

Cache stubs are {id, objectType} only and need a per-object GET to hydrate; fresh filtered results are already complete. Assets that 404 mid-hydration are dropped rather than failing the page. Accepts an optional `objectTypeId` to paginate within a single type's tab.

## updateAssetAttribute

The only portal-side write. Order of checks matters:

1. Unlicensed caller + `allowPortalEdit` off → reject.
2. `verifyAssetOwnership(objectId, requestingUser, config, workspaceId)` — always `asApp()`; rejects with a clear message if the object isn't in the caller's own ownership set. **This is the security boundary; the client-supplied `objectId` is untrusted until this passes.**
3. Write mode: `editMode === 'serviceAccount'` (default) writes `asApp()` — the only mode that works for unlicensed customers; legacy `userAccount` writes `asUser()` and is refused for unlicensed callers.
4. On success: `kvs.delete(assetListCacheKey(accountId))` so the next list load reflects the edit.

## diagnoseCaller

Read-only self-diagnostic surfaced in the portal UI when an unlicensed user sees zero assets: resolves the profile, probes workspace reachability, reads config, and counts objects in the configured schema — returning a structured report of which link in that chain is broken instead of a blank table.

---

### 3. `resolvers/assetLoadConsumer.js` — the asset-load job

---

Consumer for `asset-load-queue` (300 s budget). This is the old synchronous `getUserAssets`, relocated. Flow: config check → `resolveUser` → `resolveAssetList` (always `forceFresh`) → slice page 1 → `buildAssetPayload` with `typeIdsForColumns` = every type in the full result → compute `countsByType` / `typeCatalog` / `loadedCountsByType` (drives the per-type tabs and their "N of M" labels) → write the complete result to `jobKey(jobId)`.

Error handling contract (shared by all three consumers):

```
// Expected/handled outcomes → { status: 'done', result: { ..., error } }  
//   (matches what the old sync resolver returned – frontend shows result.error)  
// Unexpected exceptions   → { status: 'error', error } (generic failure message)  
// If even THAT write fails → frontend's bounded poll times out on its own
```

`canEdit` is computed here from the pass-through `unlicensed` flag + `config.allowPortalEdit` — not from `context`, which is untrustworthy in a consumer.

---

### 4. `resolvers/exportAssets.js` + `exportJobConsumer.js` — export

---

#### File builders (pure functions, no I/O)

| Function                                        | What it does                                                                                                                                             |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>groupAssetsByType(assets, columns)</code> | Buckets mapped assets by object type; each group carries only the columns relevant to that type; groups sorted largest-first                             |
| <code>groupToRows(group)</code>                 | Group → <code>[headerRow, ...dataRows]</code> with fixed <code>Name / Key</code> first columns                                                           |
| <code>buildXlsx(groups)</code>                  | One worksheet per type (sheet names sanitized/deduped to Excel's 31-char rules), auto column widths, returns a Buffer                                    |
| <code>buildPdf(groups, schemaName)</code>       | A4 landscape via pdfkit: manual table layout, per-row height measurement, shaded header repeated after page breaks, one type per page. Returns a Promise |
| <code>buildFilename(schemaName, format)</code>  | <code>&lt;sanitized-schema&gt;_&lt;YYYY-MM-DD&gt;.&lt;ext&gt;</code>                                                                                     |

**`startExportJob` / `getExportJobResult` (in `registerExportAssets`)**

Same async-job template as asset load. The distinctive part is chunk reassembly:

```
// consumer stored: export-job:<id> = { status, result: { ..., chunkCount } }
// export-job:<id>:chunk:0..N = base64 slices (≤90k chars each; 128KB cap)
const chunks = await Promise.all(
  Array.from({ length: chunkCount }, (_, i) => kvs.get(exportJobChunkKey(jobId, i)))
);
await Promise.all(/* delete all chunk keys + the job key */);
return { status: 'done', result: { ...meta, base64: chunks.join('') } };
```

The `invoke()` response channel has no 128 KB cap — chunking exists only to satisfy KVS while the job is parked there. Read-once semantics: polling the same finished job twice returns "not found" the second time.

#### `exportJobConsumer.handler`

Always re-runs `resolveAssetList` (fresh, filters honored, `maxUserAssetLimit` enforced) → optional in-memory narrowing to `objectTypeId` → `buildAssetPayload` → `groupAssetsByType` → `buildXlsx` / `buildPdf` → chunk & store. Never trusts a client-supplied asset array (the pre-async version did — see `docs/review.md` history note).

## 5. `resolvers/csvImport.js` + `csvImportConsumer.js` — CSV import

### Sync resolvers (agent's own session, `asUser()`)

| Resolver                            | Purpose                                                                                                                                                                                                                                               |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>getIssueCsvAttachments</code> | Lists the ticket's attachments filtered to <code>.csv</code> / <code>text/csv</code>                                                                                                                                                                  |
| <code>previewCsvImport</code>       | Downloads + parses the CSV, matches headers against the chosen object type's attributes, reports <code>matchedAttributes</code> / <code>unmatchedColumns</code> / which attribute the <b>first column</b> (the unique key, by convention) resolved to |
| <code>startCsvImportJob</code>      | Async-job template; payload = <code>{issueId, attachmentId, objectTypeId, createOnly}</code>                                                                                                                                                          |
| <code>getCsvImportJobResult</code>  | Poll; returns live progress, not just final state                                                                                                                                                                                                     |

Using `asUser()` for preview means an agent can only import from tickets/attachments they can already see.

#### `csvImportConsumer.handler` (900 s budget, `asApp()`)

Per-row pipeline (rows processed in **chunks of 10** — parallel within a chunk, sequential across chunks, to stay under Jira rate limits):



```

const processRow = async (row, rowNumber) => {
  const keyValue = String(row[uniqueKeyHeader] ?? '').trim();
  if (!keyValue) → recordError('Missing <key attr>')
  if (seenKeyValues.has(...)) → recordError('Duplicate ... within this file')

  const attributesPayload = await buildAttributesPayload(...);
  //   dates   → normalizeDateValue (ISO)
  //   objects → resolveObjectReference (display name → object id via AQL;
  //               unresolved = WARNING + attribute dropped, row still imports)
  //   rest     → raw string

  const existing = first result of
    `objectTypeId = T AND objectSchemaId = S AND "<keyAttr>" = "<keyValue>"`;

  if (existing && createOnly) → recordError('already exists - not overwritten')
  if (existing && no value differs from stored) → summary.unchanged++ // no PUT
  if (existing) → PUT /object/{id} → summary.updated++
  else → POST /object/create → summary.created++
};

```

Details that answer the "why" questions:

- **The queue payload carries `attachmentId`, not parsed rows** — queue/KVS payloads have size limits and re-parsing is cheap; it also guarantees preview and import read the same bytes.
- **`resolveObjectReference` cache is per-invocation** (a `Map` created inside `handler`), not module-scope — a warm Lambda container could otherwise serve a stale id across unrelated jobs after a referenced object was renamed/recreated.
- **Unchanged detection** compares the computed payload against `existing.attributes` via `getAttrValue(attr, raw=true)` — both sides are in "raw" shape (ISO dates, object ids), so plain string comparison is valid. This makes re-running the same CSV idempotent and the summary honest.
- **Progress is re-written to KVS after every chunk** — that's what the panel's progress bar polls.

---

## 6. `resolvers/adminResolvers.js` — admin page

---

All `asUser()` — these run from the admin page, so the admin's own permissions apply. All are read-only against Jira except `saveConfig` / `applyReconciliation` (which write only to KVS).

| Resolver                             | Purpose                                               | Notes                                                                                                                              |
|--------------------------------------|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <code>getSchemas</code>              | List Assets schemas for the picker                    |                                                                                                                                    |
| <code>getObjectTypes</code>          | Types for the selected schema                         | <code>excludeAbstract=false</code> — abstract parents are listed too                                                               |
| <code>getObjectTypeAttributes</code> | Per-type attribute defs for the visibility checklists | Filters built-ins; returns <code>{attributeId, attributeName, attributeType, isEditable, options}</code>                           |
| <code>saveConfig</code>              | Sanitize + <b>full-replace</b> the config in KVS      | Bumps <code>version</code> ; malformed <code>hiddenByObjectType</code> / <code>aqlRows</code> shapes are coerced, not rejected     |
| <code>validateAql</code>             | "Test this ownership rule" button                     | Builds the AQL for a rule + a sample user and runs it, returning count or error; a 0-count returns a <i>warning</i> , not an error |
| <code>searchAssetUsers</code>        | Admin preview: find a "User" object by name           | Backs the UserPicker-style preview flow                                                                                            |
| <code>getAssetsForUser</code>        | Admin preview: what would user X see?                 | Same pipeline as the portal but <code>ignoreHidden: true</code> so admins see everything                                           |
| <code>reconcileConfig</code>         | Detect config↔schema drift                            | See below                                                                                                                          |
| <code>applyReconciliation</code>     | Strip confirmed-ghost ids from saved config           | Only removes what <code>reconcileConfig</code> confirmed; unverified types untouched                                               |

### `reconcileConfig` — the drift detector

Compares saved `hiddenByObjectType` keys/ids against the live schema. The part that matters:

```
const RECONCILE_CHUNK_SIZE = 5;           // bounded concurrency...
const fetchFailedTypeIds = new Set();      // ...and explicit failure tracking
// ...
if (fetchFailedTypeIds.has(typeId)) continue; // unverified ≠ deleted
```

A rate-limited attribute fetch used to be silently treated as "this type has zero attributes," flagging every hidden attribute on it as a ghost — a false "Schema drift detected" banner that worsened with schema size, and whose "Clean up" button would then really delete valid config. Failed fetches are now reported as `unverifiedObjectIds` and excluded from ghost detection; the UI shows them as "couldn't verify" instead of "deleted."

## 7. Adding a new resolver — checklist

---

1. Define it in the appropriate file ( `resolver.define('name', async ({ payload, context }) => ...)` ); register via the existing `register*` pattern if it's a new file.
2. Decide auth: does the caller's own permission suffice ( `asUser()` )? Could an unlicensed portal user hit it (check `isUnlicensedCaller(context)` and use `asApp()` + explicit authorization)?
3. Any user-controlled string entering AQL → `escapeAqlValue` . Attribute references in AQL are **names**, never numeric ids.
4. Could it take > ~10 s worst-case? Use the async job template ( `startAssetLoadJob` is the reference implementation) — new queue + consumer + function in `manifest.yml` , thin re-export in `src/` , and remember `forge install --upgrade` after adding a module.
5. Writing anything to KVS? Check the 128 KB cap. Changing a user's assets?  
`kvs.delete(assetListCacheKey(accountId))` .
6. `forge lint` → `deploy` → verify via `forge logs` .